

Solids and Creatures

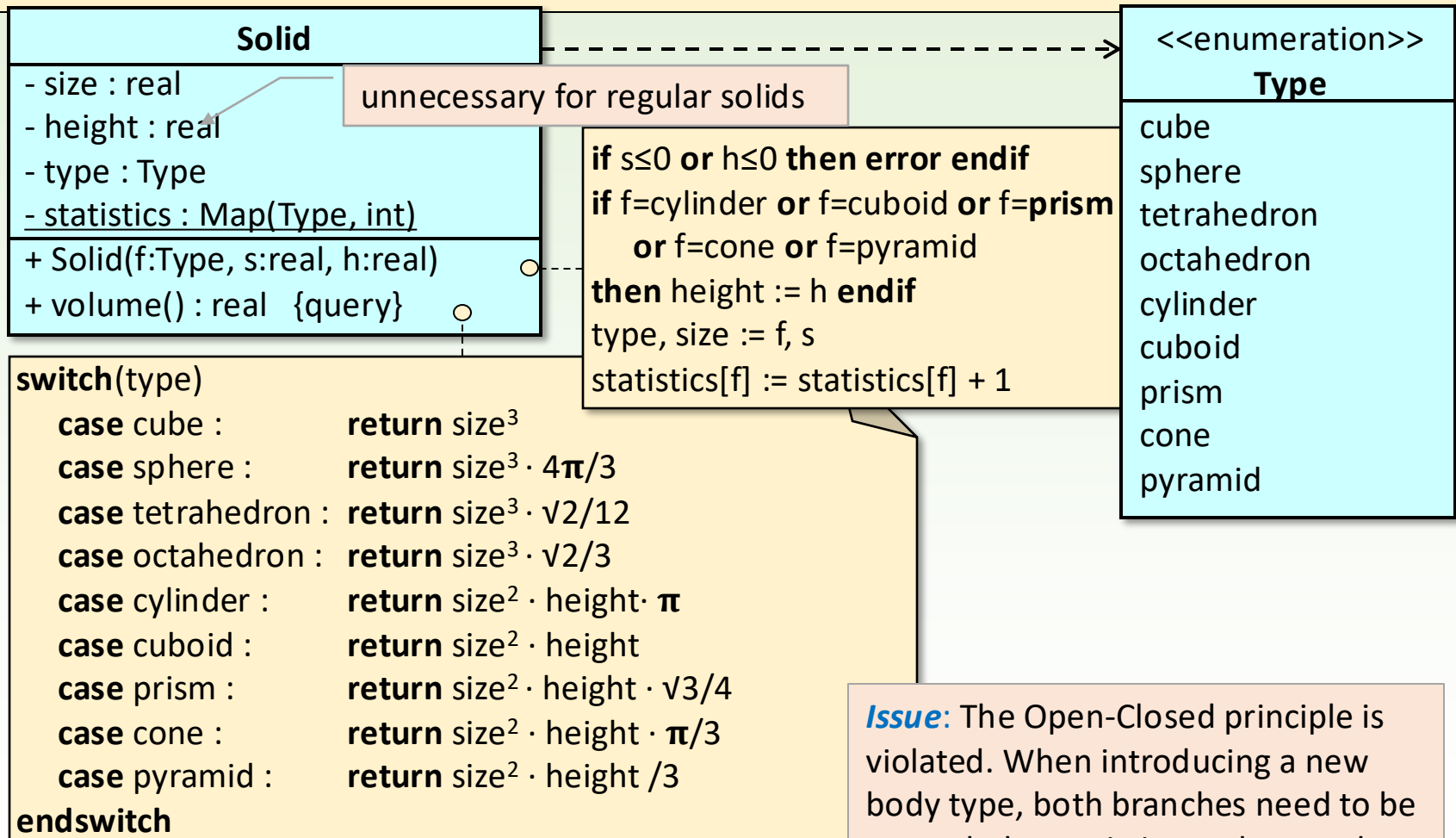
1st Part

Volume of Bodies

Gregorics Tibor and Pintér Balázs

Task and its awkward model

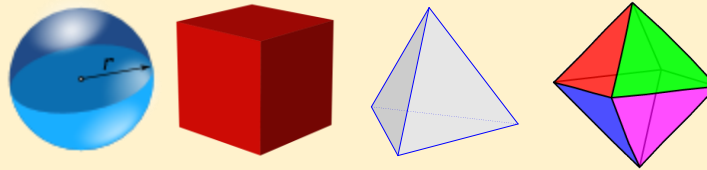
Let's create a program that allows us to create various geometric solids so that we can calculate their volume. We should be able to generate statistics at any time about how many instances of each type of solid were created.



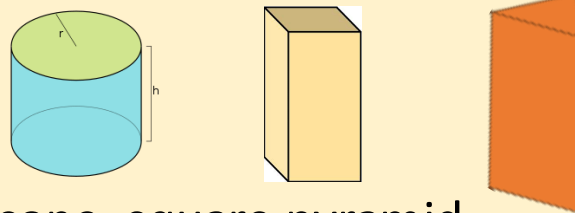
Object hierarchy

Solids can be categorized in the following way:

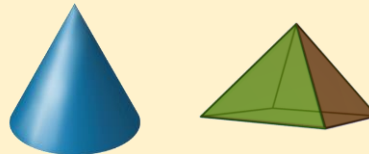
- regular solids: sphere, cube, tetrahedron, octahedron



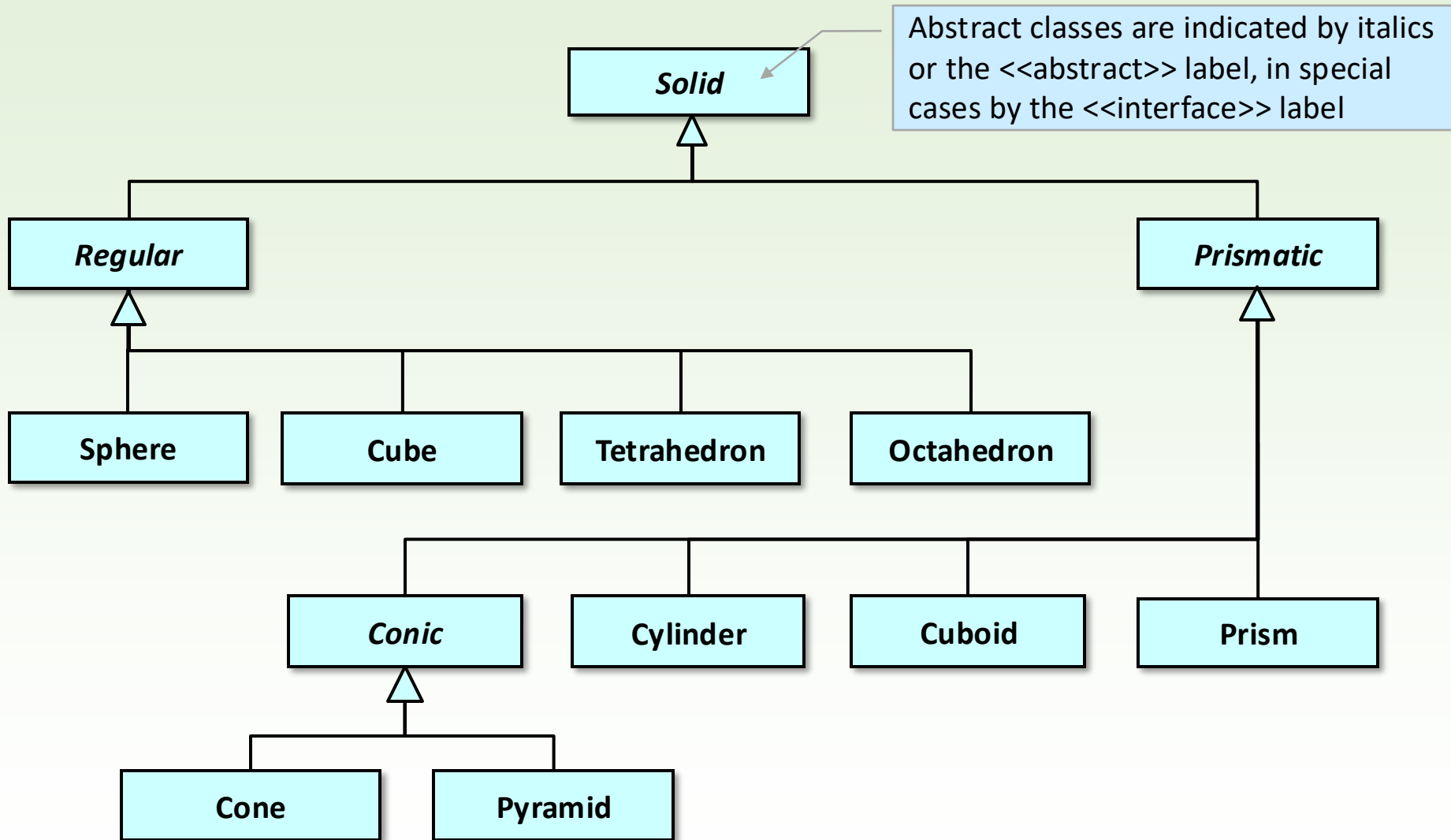
- prismatic solids: cylinder, square and regular triangular prism



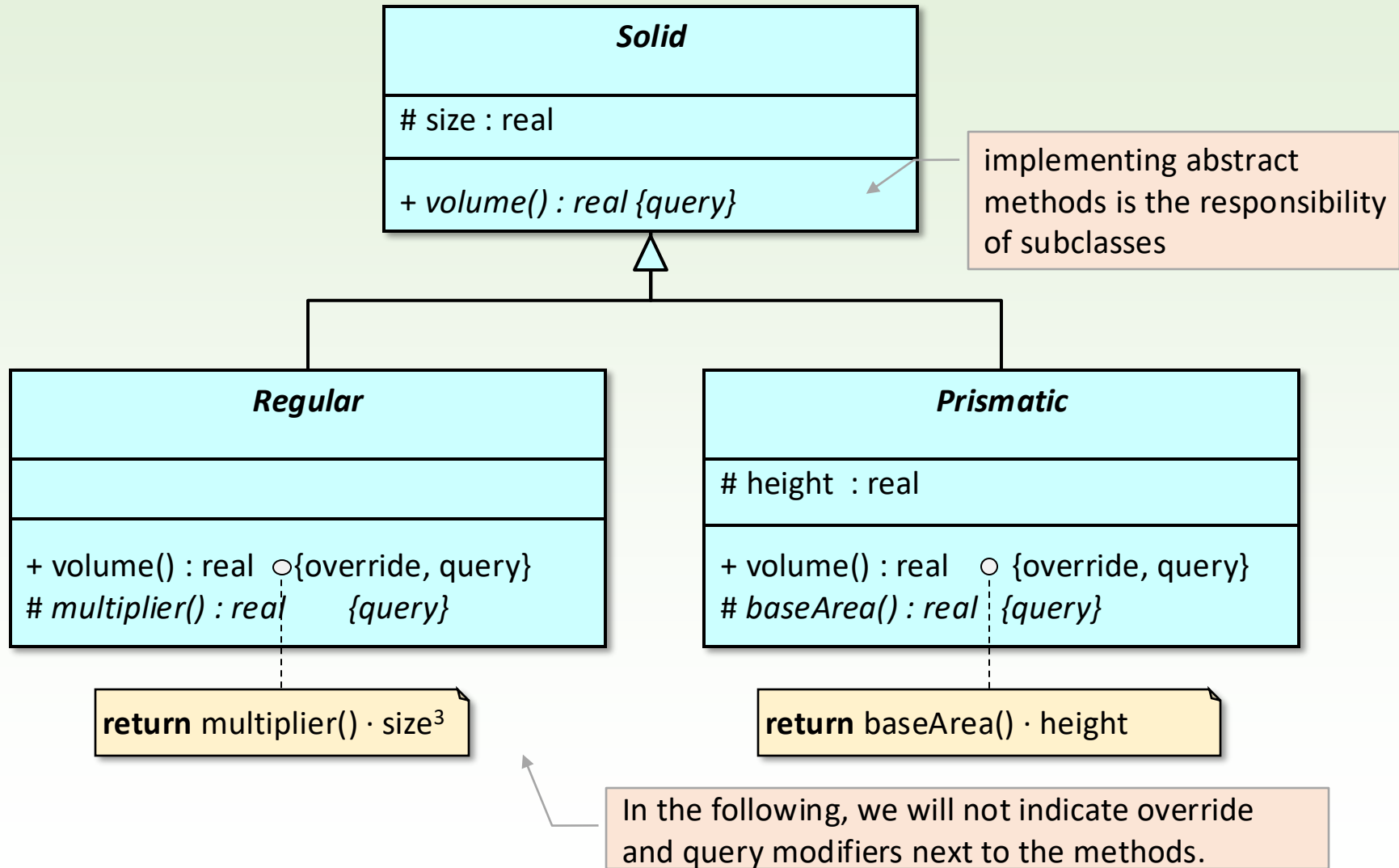
- conic solids: cone, square pyramid



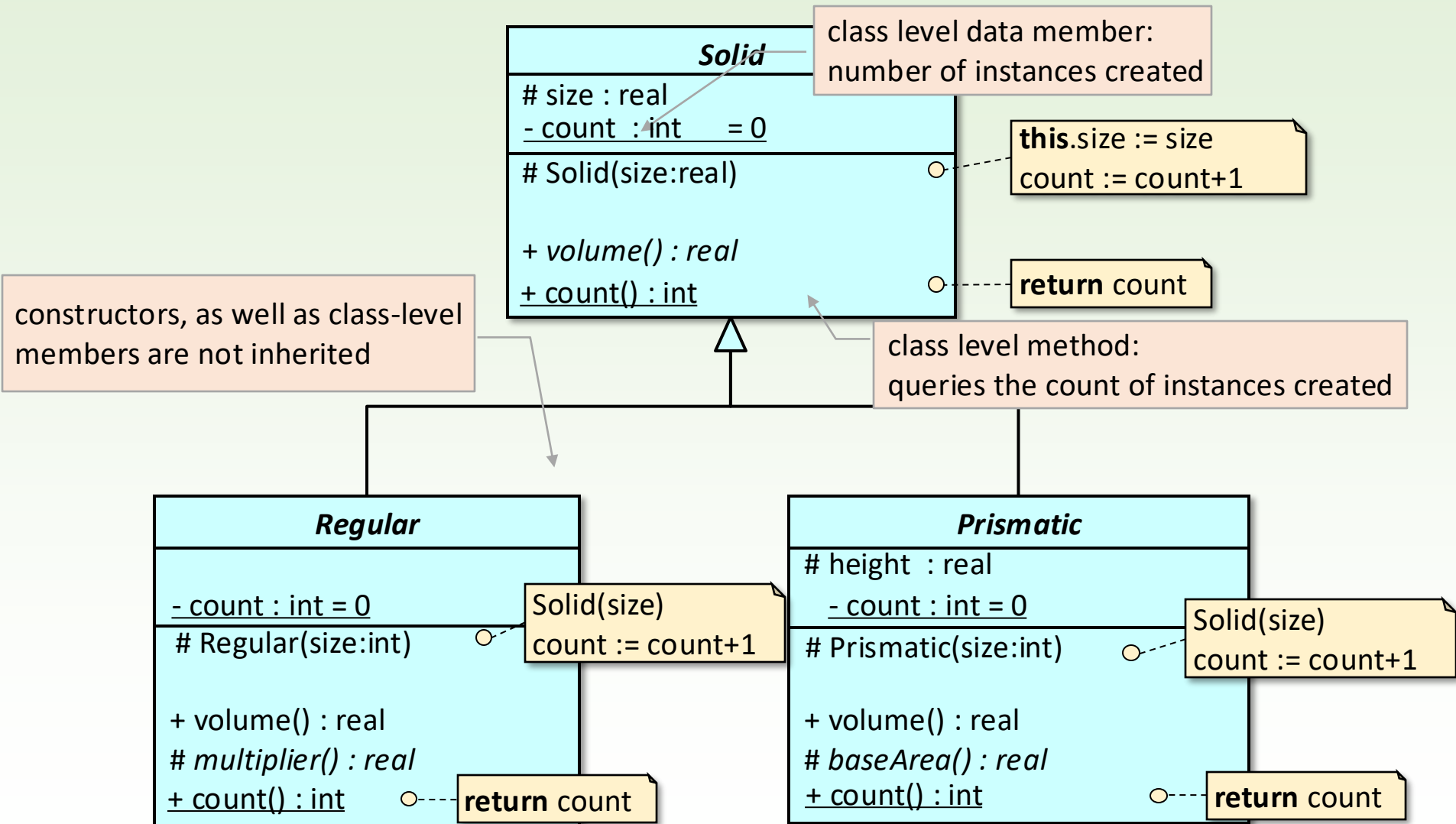
Class hierarchy



Classes of abstract solids



Constructors, class-level members



The base class of solids

```
public abstract class Solid
```

abstract class, because
its constructor is not public, and
it has an abstract method (volume)

```
{
```

```
protected double size;
```

class-level data member

```
private static int count = 0;
```

it runs when an object derived from
Solid is created

```
protected Solid(double size) {
```

```
    this.size = size;
```

```
    ++count;
```

```
}
```

abstract method

```
public abstract double volume();
```

class-level method

```
public static int getCount() { return count; }
```

```
}
```

Class of regular abstract solids

```
public abstract class Regular extends Solid
{
    private static int count = 0;

    protected Regular(double size) {
        super(size);
        ++count;
    }

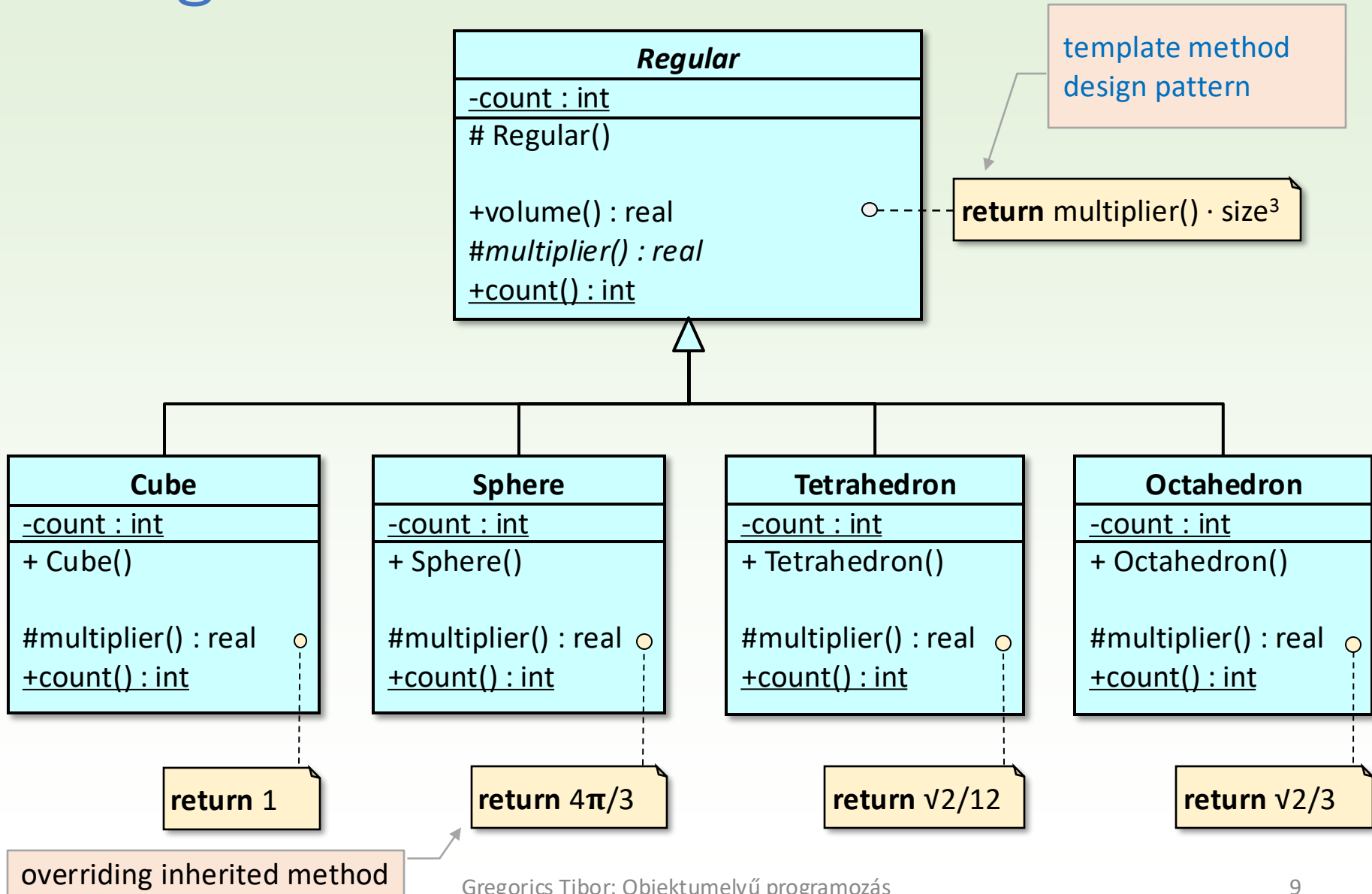
    @Override
    public double volume() {
        return size * size * size * multiplier();
    }

    protected abstract double multiplier();

    public static int getCount() {
        return count;
    }
}
```

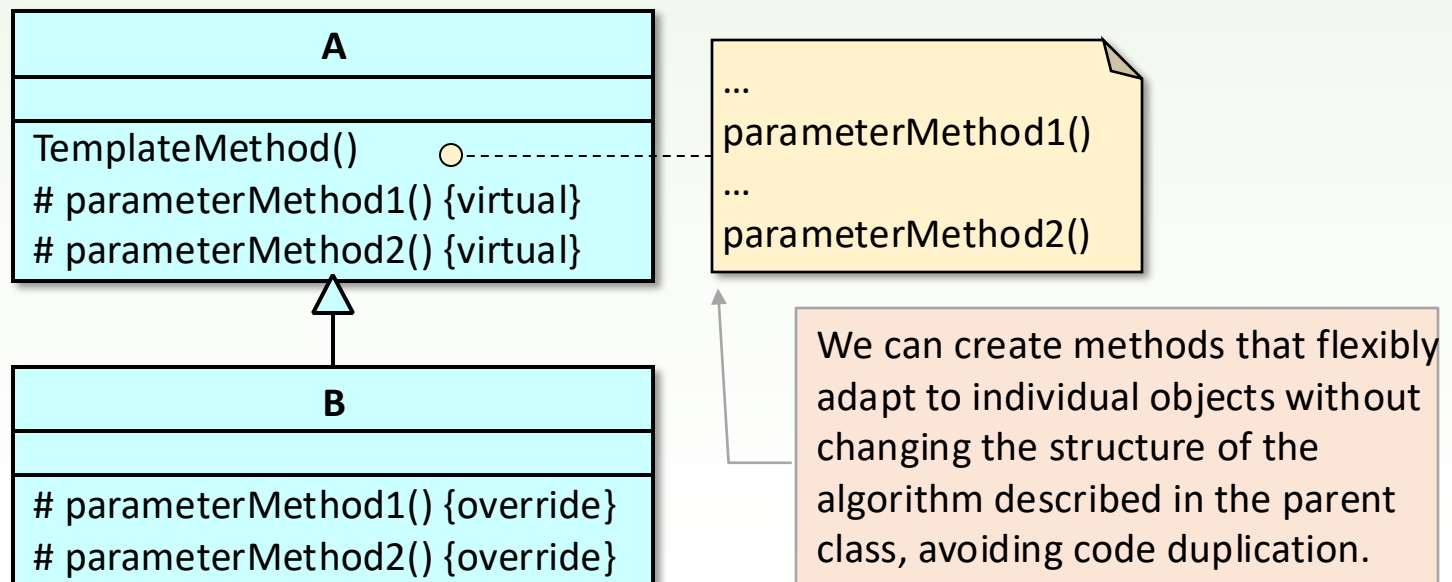
super: calls constructor of parent class
with specific parameters

Regular solids



Template method design pattern

- We define an activity (template method) as a method of a parent class in such a way that its special, subtype-dependent parts are only indicated as abstract method calls (template parameters). Then we override these methods in the subclasses in the way expected there.



Sphere

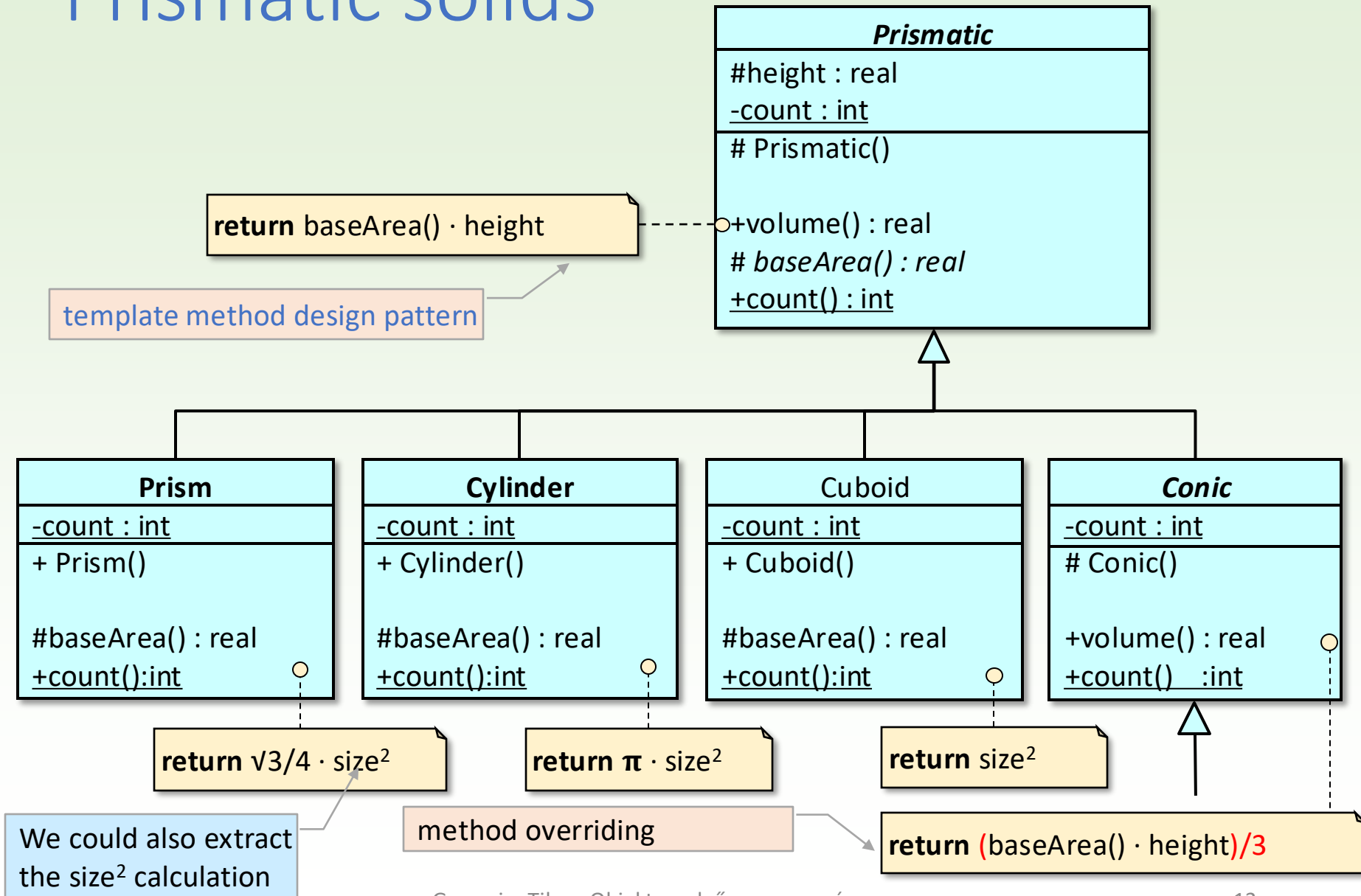
```
public class Sphere extends Regular
{
    private static int count = 0;

    public Sphere(double size)
    {
        super(size);
        ++count;
    }

    @Override
    protected double multiplier()
    {
        return (4.0 * Math.PI) / 3.0;
    }

    public static int getCount()
    {
        return count;
    }
}
```

Prismatic solids



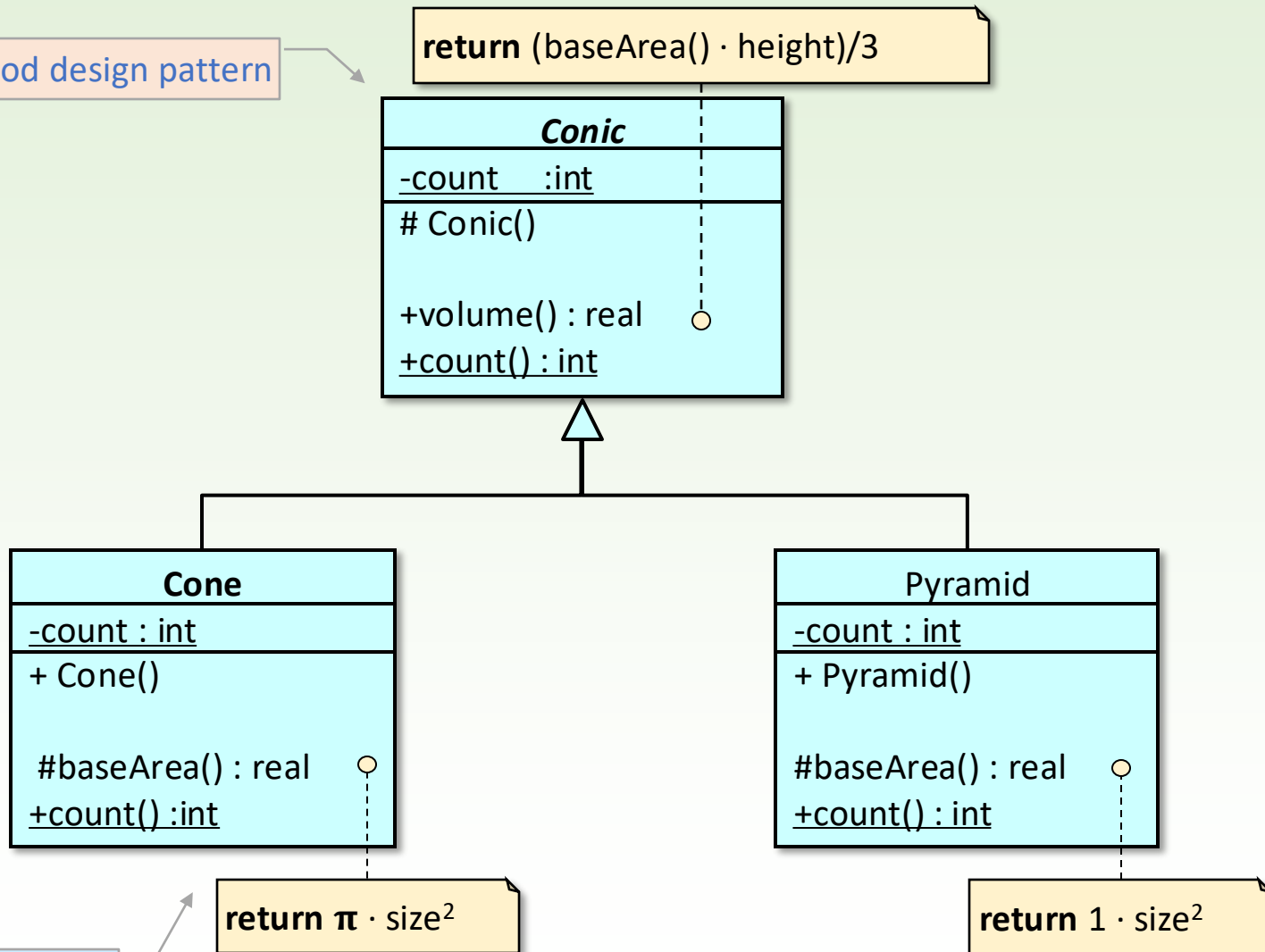
Cylinder

```
public abstract class Prismatic extends Solid {  
    protected double height;  
    private static int count = 0;  
  
    public Prismatic(double size, double height) {  
        super(size);  
        this.height = height;  
        ++count;  
    }  
    public double volume() { return area() * height; }  
  
    protected abstract double area();  
  
    public static int count() { return count; }  
}
```

```
public class Cylinder extends Prismatic {  
  
    private static int count = 0;  
    public Cylinder(double size, double height) {  
        super(size, height);  
        ++count;  
    }  
    @Override  
    protected double area() {  
        return 3.14159 * size * size;  
    }  
    public static int getCount() {  
        return count;  
    }  
}
```

Conic solids

template method design pattern



We could also extract the size^2 calculation

Main program

Cube 5.0
Cylinder 3.0 8.0
Cylinder 1.0 10.0
Tetrahedron 4.0
SquarePyramid 3.0 10.0
Octahedron 1.0
Cube 2.0
SquarePyramid 2.0 10.0

solids.txt

```
public static void main(String[] args)
{
    try (Scanner scanner = new Scanner(new File("solids.txt"))) {
        List<Solid> solids = new ArrayList<>();

        Solid sh;
        while ((sh = Solid.create(scanner)) != null) {
            solids.add(sh);
        }

        statistics();
        System.out.println("Volumes:");
        for (Solid solid : solids) {
            System.out.printf("%s : %.2f%n",
                solid.getClass().getSimpleName(),
                solid.volume());
        }
    } catch (FileNotFoundException e) {
        System.err.println("File not found: " + e.getMessage());
    }
}
```

Creating instances of various objects based on the lines of a text file.

Printing the volume of solids

printing of the number of distinct solids

the type name

Due to runtime polymorphism, the volume() method of the specific object referenced by solid is called, not the one from the Solid base class.

Instantiation of Solids

Cube 5.0 [solids.txt](#)

Due to inheritance, a variable of type Solid can be assigned a value (reference) of type

class-level factory method

Octahedron 1.0

Cube 2.0

SquarePyramid 2.0 10.0

```
public static Solid create(Scanner scanner)
{
    public static Solid create(Scanner scanner) {
        if (!scanner.hasNext()) return null;

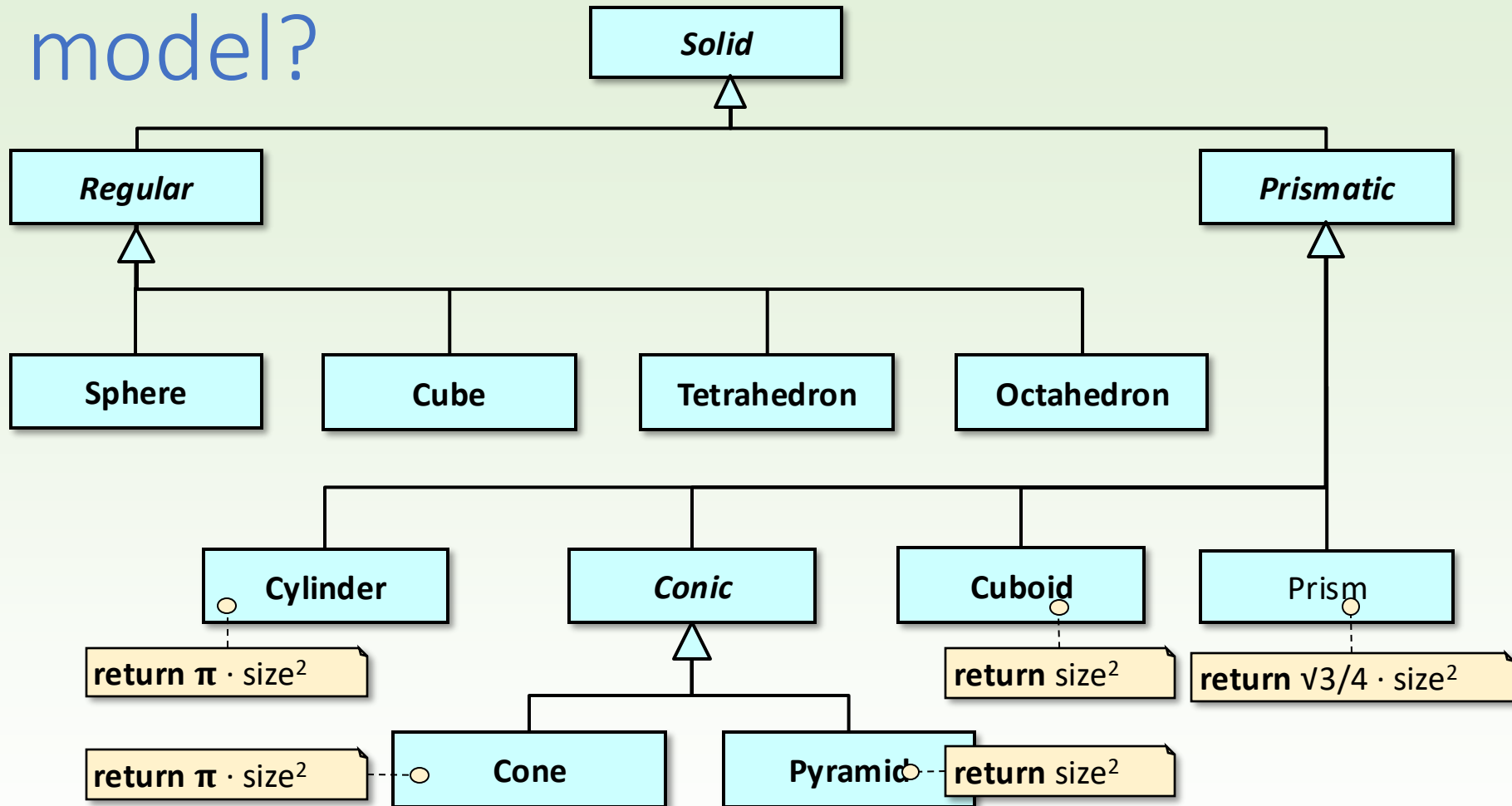
        String type = scanner.next();
        double size = scanner.nextDouble();

        try {
            // Using modern Java switch expression with arrows
            return switch (type) {
                case "Cube" -> new Cube(size);
                case "Sphere" -> new Sphere(size);
                case "Tetrahedron" -> new Tetrahedron(size);
                case "Octahedron" -> new Octahedron(size);
                case "Cylinder" -> {
                    double height = scanner.nextDouble();
                    yield new Cylinder(size, height);
                }
                case "SquarePrism" -> {
                    double height = scanner.nextDouble();
                    yield new SquarePrism(size, height);
                }
            };
        }
    }
}
```

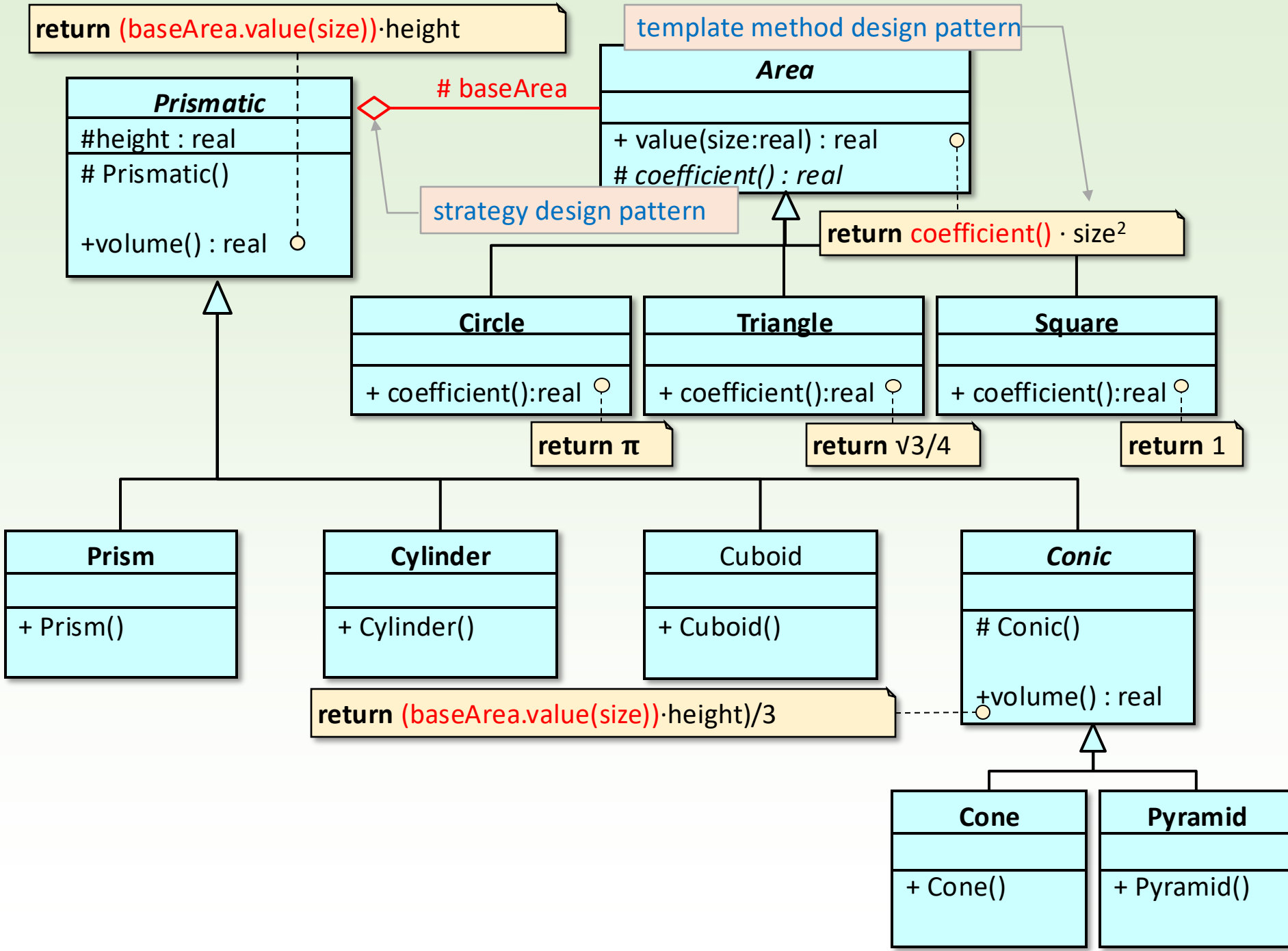
```
case "TriangularPrism" -> {
    double height = scanner.nextDouble();
    yield new TriangularPrism(size, height);
}
case "Cone" -> {
    double height = scanner.nextDouble();
    yield new Cone(size, height);
}
case "SquarePyramid" -> {
    double height = scanner.nextDouble();
    yield new SquarePyramid(size, height);
}
default -> throw new UnknownSolidException();
};

} catch (UnknownSolidException e) {
    return null;
}
}
```


Is there any redundancy left in the model?



Issue with the model: Redundancy (code duplication) has appeared. The calculation of the same base area is present in multiple classes: the circle's in the cone and cylinder classes, and the square's in the square-based prism and pyramid classes.



Cylinder

```
public abstract class Prismatic extends Solid {
    protected IArea baseArea;
    protected double height;
    private static int count = 0;

    protected Prismatic(double size, double height) {
        super(size);
        this.height = height;
        ++count;
    }

    @Override
    public double volume() {
        return baseArea.area(size) * height;
    }

    public static int getCount() {
        return count;
    }
}
```

a data member suitable for storing the reference to the Area object

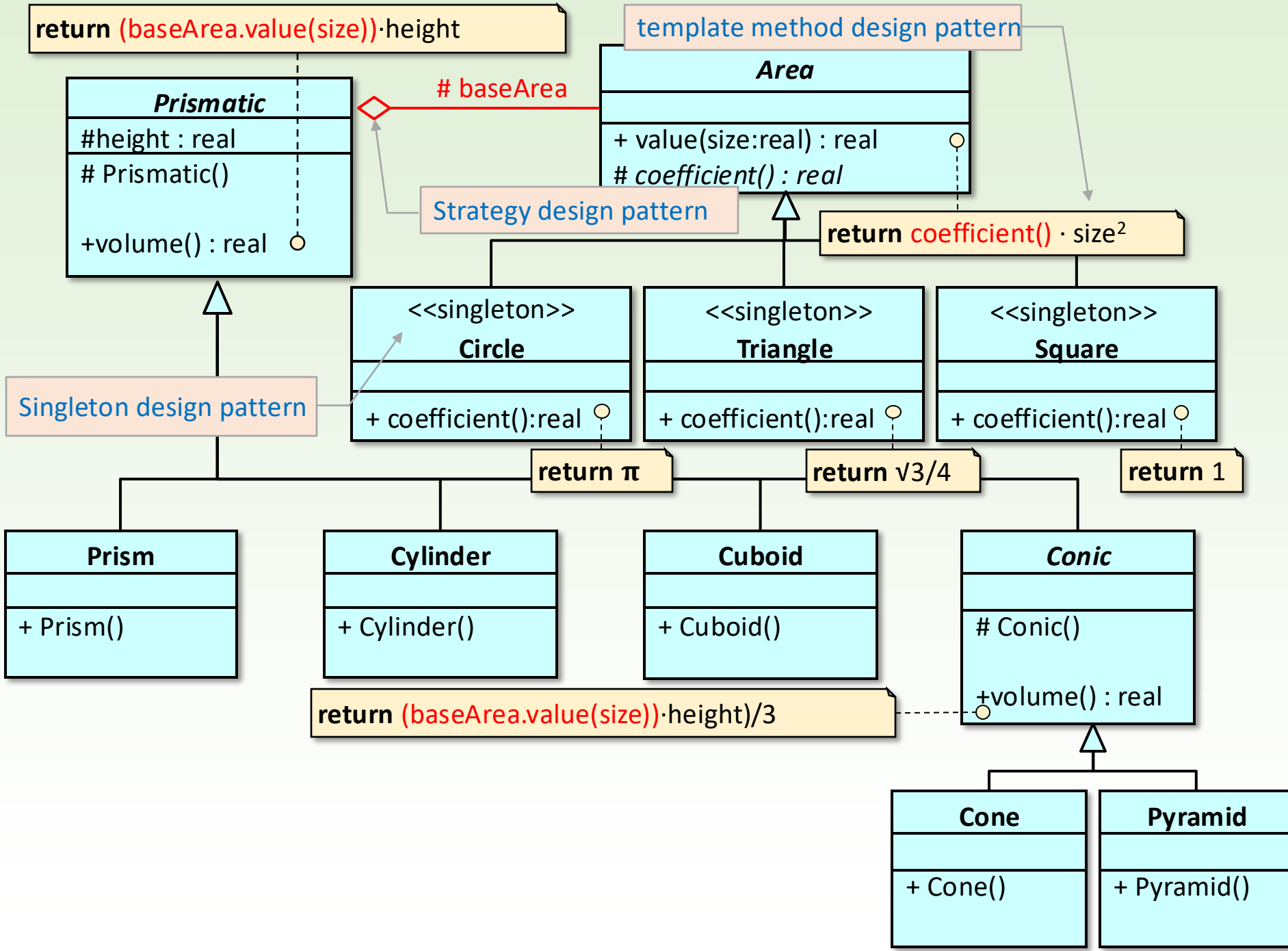
the Area object's method for calculating area

```
public class Cylinder extends Prismatic {
    private static int count = 0;

    public Cylinder(double size, double height) {
        super(size, height);
        baseArea = new CircleArea();
        ++count;
    }

    public static int getCount() {
        return count;
    }
}
```

creation and injection of Circle area calculator object



Area Singletons

```
public class CircleArea implements IArea {  
    private static CircleArea instance;  
  
    private CircleArea() { }  
  
    @Override  
    public double area(double r) {  
        return Math.PI * r * r;  
    }  
  
    public static CircleArea getInstance() {  
        if(instance == null) instance = new CircleArea();  
        return instance;  
    }  
}
```

```
public class Cylinder extends Prismatic {  
    private static int count = 0;  
  
    public Cylinder(double size, double height) {  
        super(size, height);  
        baseArea = CircleArea.getInstance();  
        ++count;  
    }  
  
    public static int getCount() {  
        return count;  
    }  
}
```

Creating and injecting a circle area calculator object

Solids and Creatures

2nd part

Creature survival competition

Gregorics Tibor and Pintér Balázs

Task

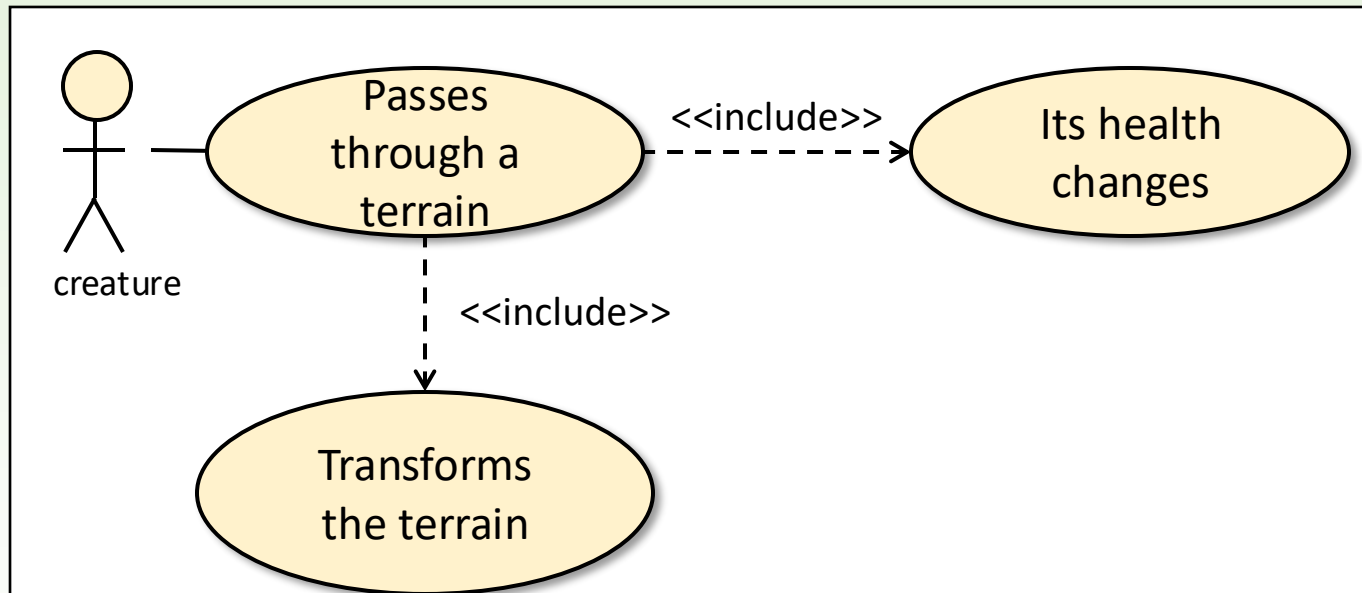
Let's simulate a survival competition among various creatures.

Creatures belong to one of three species (Beetles, Splashers, Greenies). They have a name, a species, and their current health points. They proceed one after another along a course with three types of terrain (sandy, grassy, swampy). When a creature traverses a terrain section, it transforms the terrain depending on its species (and the terrain type), while its health points change. If its health points run out, the creature dies. List the names of the creatures that survive to the end of the course!

- **Beetle:** on grass, its health decreases by two, on sand it increases by three, in swamp it decreases by four; it transforms grass into sand and swamp into grass, but does not change sand.
- **Splasher:** Its health decreases by two on grass and by five on sand, but increases by six in swamp; it transforms grass into swamp but does not change the other two terrain types.
- **Greenie:** its health increases by one on grass, decreases by two on sand, decreases by one in swamp; it transforms swampy terrain into grass, but does not change the other two types of terrain.



One step of a creature



What happens when a creature passes through a terrain?



beetle	health change	terrain change
grass	-2	sand
sand	+3	-
swamp	-4	grass

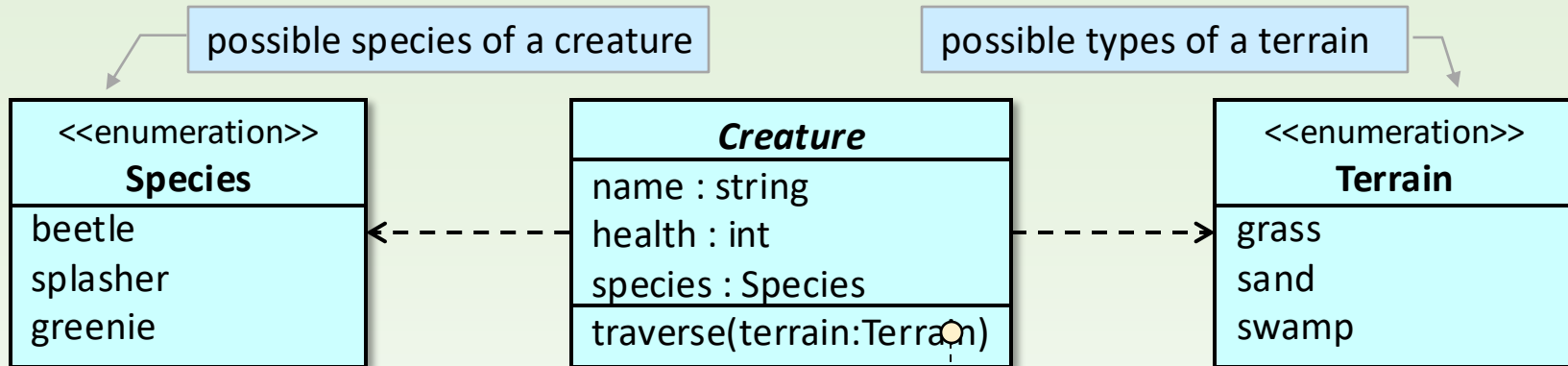


splasher	health change	terrain change
grass	-2	swamp
sand	-5	-
swamp	+6	-



greenie	health change	terrain change
grass	+1	-
sand	-2	-
swamp	-1	grass

Class diagram: 1st attempt

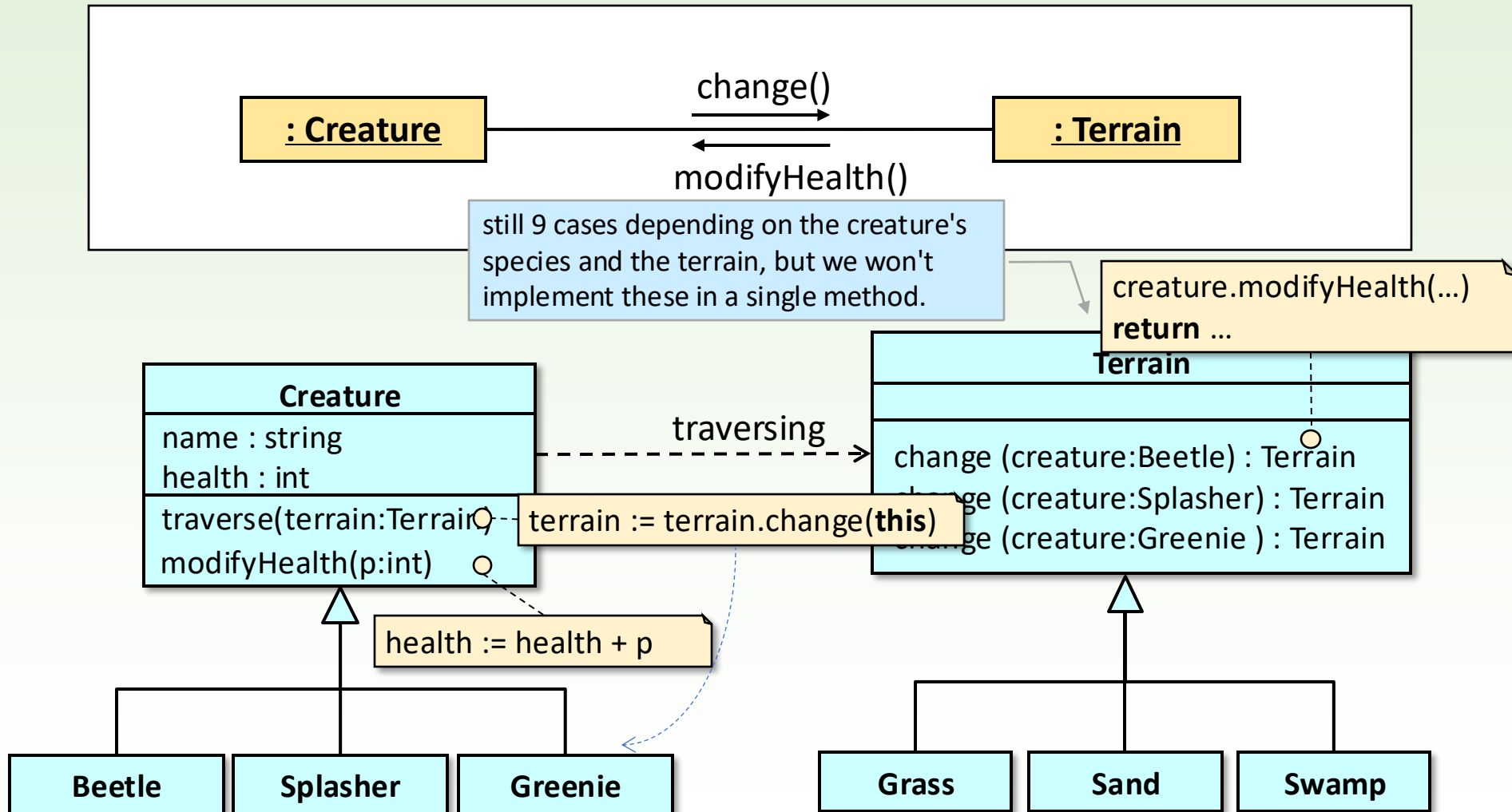


```
if species=beetle and terrain = grass then health := health-2; terrain := sand;
elseif species=beetle and terrain = sand then health := health+3;
elseif species=beetle and terrain=swamp then health := health-4; terrain := grass;
elseif species=splasher and terrain = grass then health := health-2; terrain := swamp;
elseif species=splasher and terrain = sand then health := health-5;
elseif species=splasher and terrain=swamp then health := health+6;
elseif species=greenie and terrain = grass then health := health+1;
elseif species=greenie and terrain = sand then health := health-2;
elseif species=greenie and terrain=swamp then health := health-1; terrain := grass;
endif
```

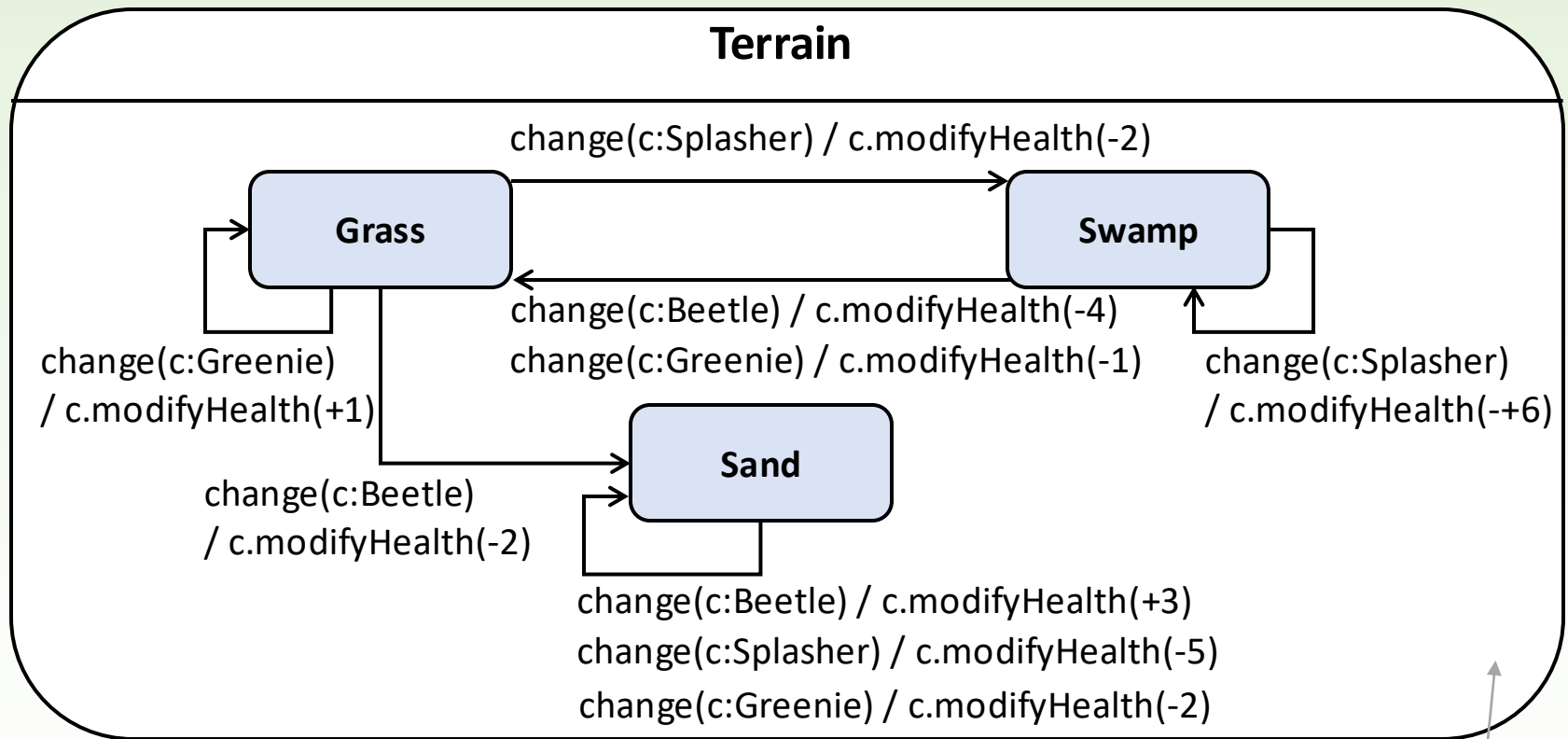
9 cases need to be described
depending on the creature and
the terrain

Issue: The Open-Closed principle is violated when introducing a new terrain type as the branching must be expanded, so existing code needs to be modified

Class diagram: 2nd attempt

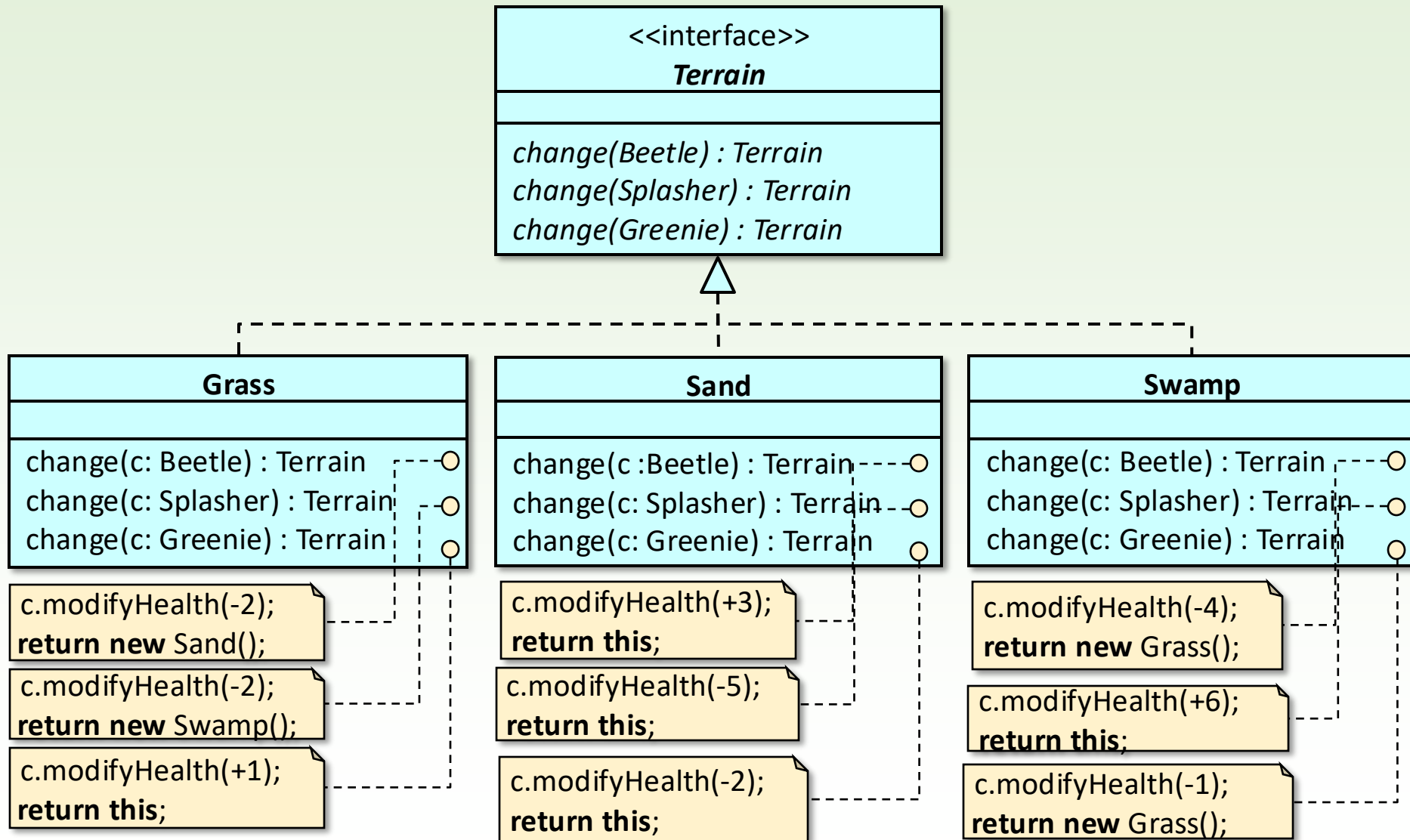


Changes in the state of a terrain



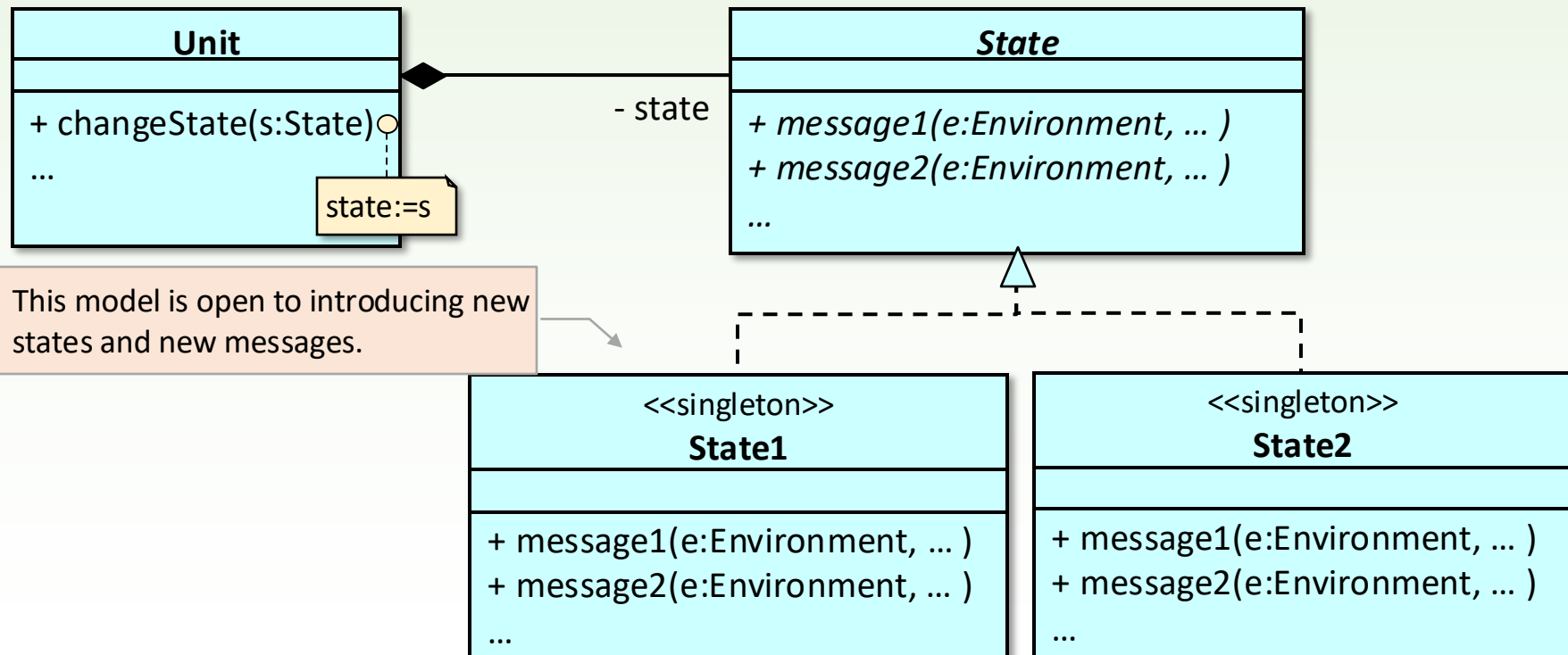
When implementing the state machine, the `change()` method of the `Terrain` class would contain a 9-branch conditional statement. This does not conform to the Open-Closed principle. It would be better to introduce 3 `change()` methods in each subclass of the `Terrain` class, which would be distinguished by having different signatures, as they would receive creatures of different Species (types) as parameters.

Classes of terrains

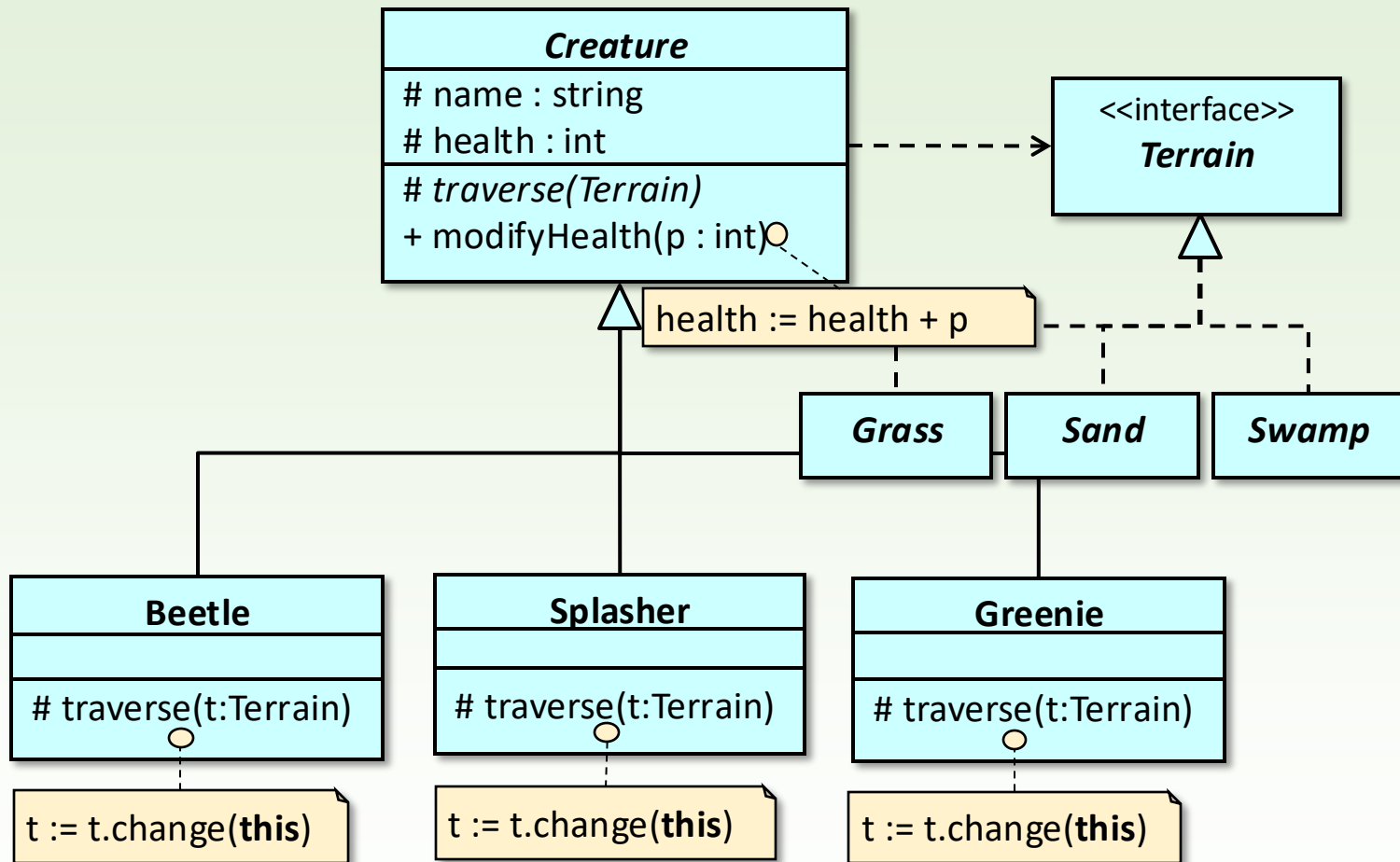


State Design Pattern

- When an object's methods behave differently depending on the object's state, instead of repeating the same conditionals, we organize the object's states into separate (possibly singleton) objects. These share a common interface that specifies the state-dependent methods, while the original object maintains the state-changing method.



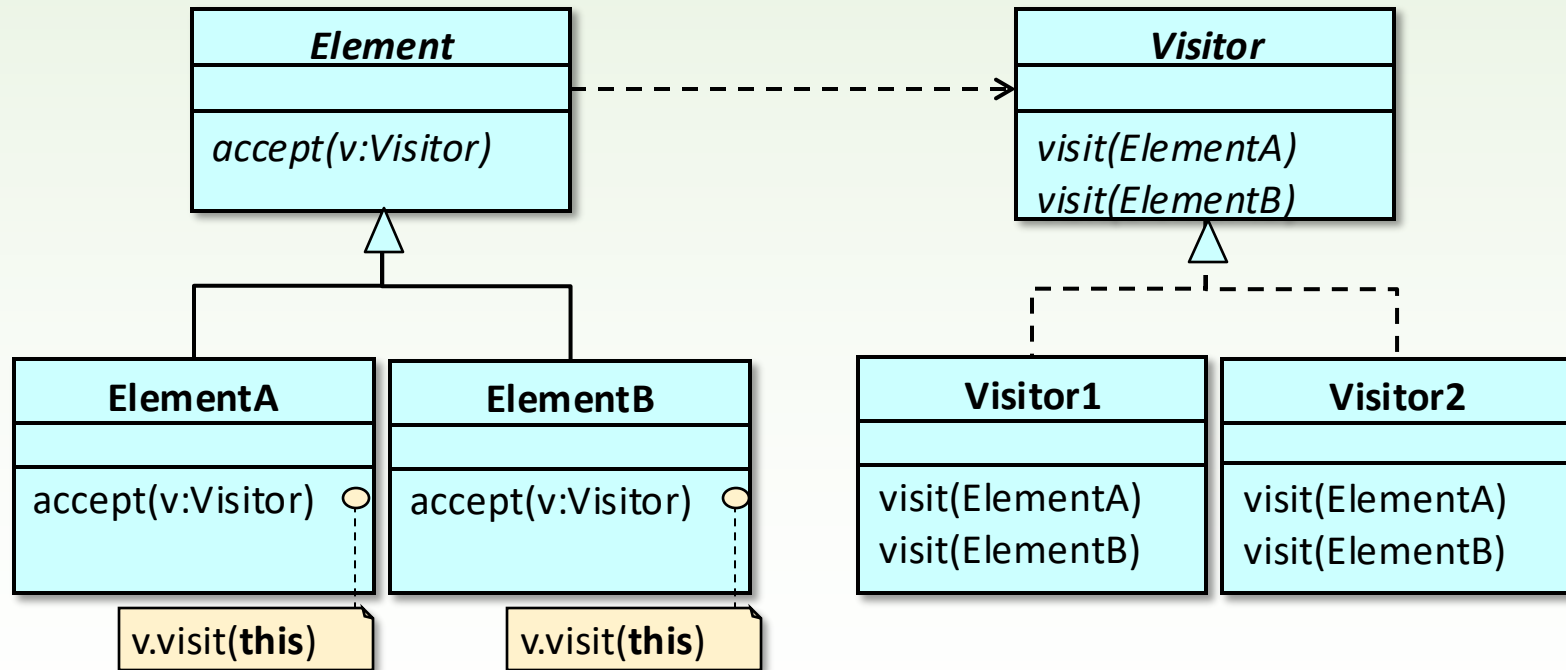
Creature classes



it was necessary to override the `traverse()` method so that the type of **this** could be read from it at compile time, since the `change()` method can only be overloaded in single dispatch languages like Java

Visitor design pattern

- When a method's (`accept()`) operation depends not only on its own class but also on which subclass of a class its parameter refers to, and we don't want this dependency to appear in the code as a conditional statement (or conditional statements with the same structure).



subtype and ad hoc
polymorphism

Classes of terrain types

```
public class Grass implements ITerrain
{
    public ITerrain change(Beetle p) { p.modifyHealth(-2); return new Sand(); }
    public ITerrain change(Splasher p) { p.modifyHealth(-2); return new Swamp(); }
    public ITerrain change(Greenie p) { p.modifyHealth(1); return this; }
}
```

```
public class Sand implements ITerrain
{
    public ITerrain change(Greenie p) { p.modifyHealth(-2); return this; }
    public ITerrain change(Beetle p) { p.modifyHealth(3); return this; }
    public ITerrain change(Splasher p) { p.modifyHealth(-5); return this; }
}
```

```
public class Swamp implements ITerrain
{
    public ITerrain change(Greenie p) { p.modifyHealth(-1); return new Grass(); }
    public ITerrain change(Beetle p) { p.modifyHealth(-4); return new Grass(); }
    public ITerrain change(Splasher p) { p.modifyHealth(6); return this; }
}
```

Issue of efficiency:

Change() always instantiates a new terrain object whenever a terrain changes, even though a single object for each terrain type would be sufficient. It could then be referenced from multiple locations.

The terrains should be singletons!

```
public interface ITerrain {
    ITerrain change(Beetle p);
    ITerrain change(Splasher p);
    ITerrain change(Greenie p);
}
```

interface

Our terrain classes are singletons

```
public class Grass implements ITerrain {  
    private static Grass instance = null;  
    private Grass() { }  
    public static Grass getInstance() {  
        if (instance == null) {  
            instance = new Grass();  
        }  
    }  
}
```

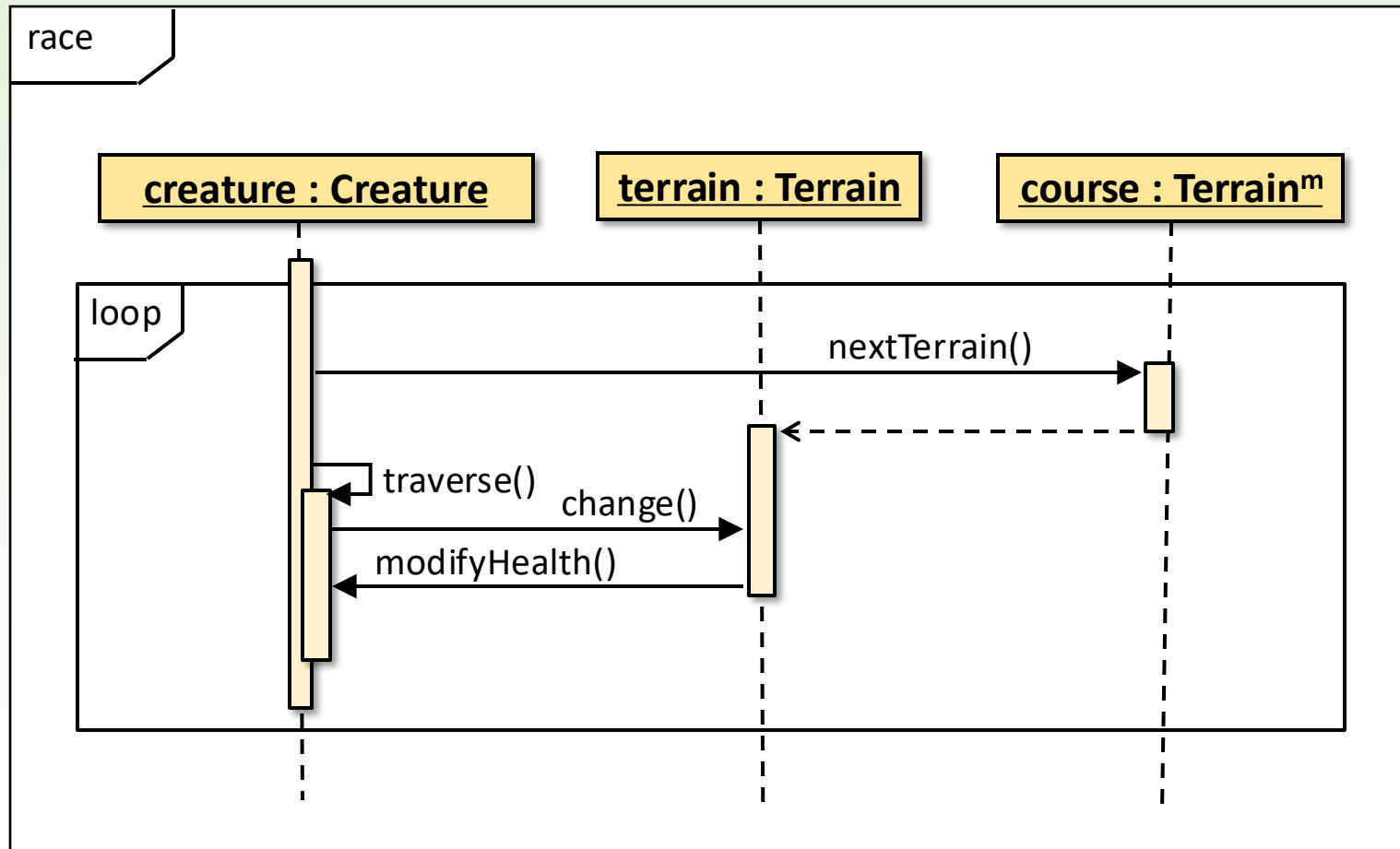
```
public class Sand implements ITerrain {  
    private static Sand instance = null;  
    private Sand() { }  
    public static Sand getInstance() {  
        if (instance == null) {  
            instance = new Sand();  
        }  
    }  
}
```

```
public class Swamp implements ITerrain {  
    private static Swamp instance = null;  
    private Swamp() { }  
    public static Swamp getInstance() {  
        if (instance == null) {  
            instance = new Swamp();  
        }  
        return instance;  
    }  
}
```

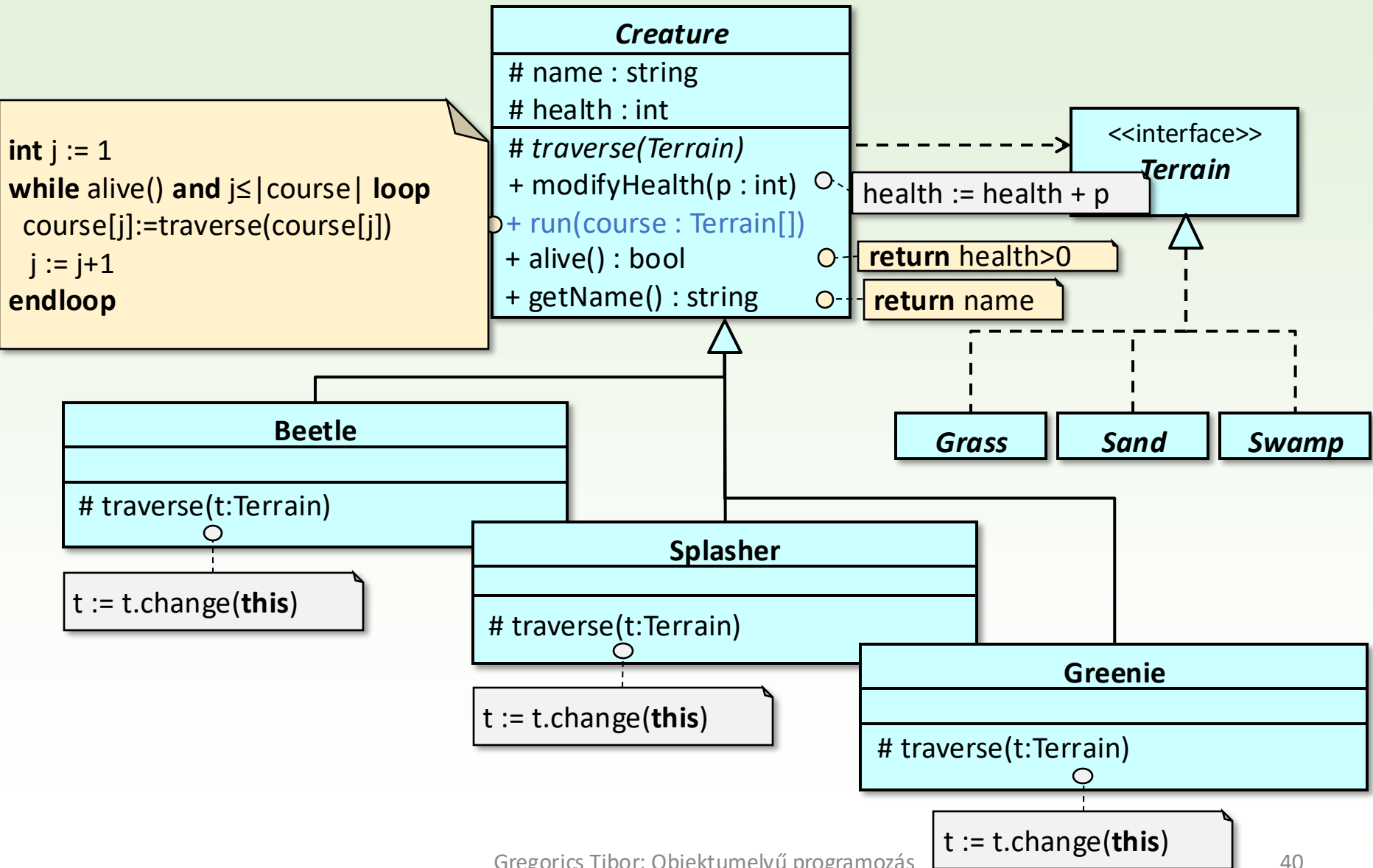
```
@Override  
public ITerrain change(Greenie p) { p.modifyHealth(-1); return Grass.getInstance(); }  
@Override  
public ITerrain change(Beetle p) { p.modifyHealth(-4); return Grass.getInstance(); }  
@Override  
public ITerrain change(Splasher p) { p.modifyHealth(6); return this; }  
}
```

```
public interface ITerrain {  
    ITerrain change(Beetle p);  
    ITerrain change(Splasher p);  
    ITerrain change(Greenie p);  
}
```

A creature's race



More methods in the Creature class



```

public abstract class Creature {
    private final String name;
    protected int health;
    public String getName() { return name; }
    public void modifyHealth(int e) { health += e; }
    public boolean isAlive() { return health > 0; }

    protected Creature(String name, int health)
    {
        this.name = name;
        this.health = health;
    }
    public void race(List<ITerrain> course)
    {
        for (int j = 0; isAlive() && j < course.size(); j++) {
            course.set(j, traverse(course.get(j)));
        }
    }
    protected abstract ITerrain traverse(ITerrain ground);
}

```

```

public class Beetle extends Creature {
    public Beetle(String name, int health) { super(name, health); }
}

```

```

public class Splasher extends Creature {
    public Splasher(String name, int health) { super(name, health); }
}

```

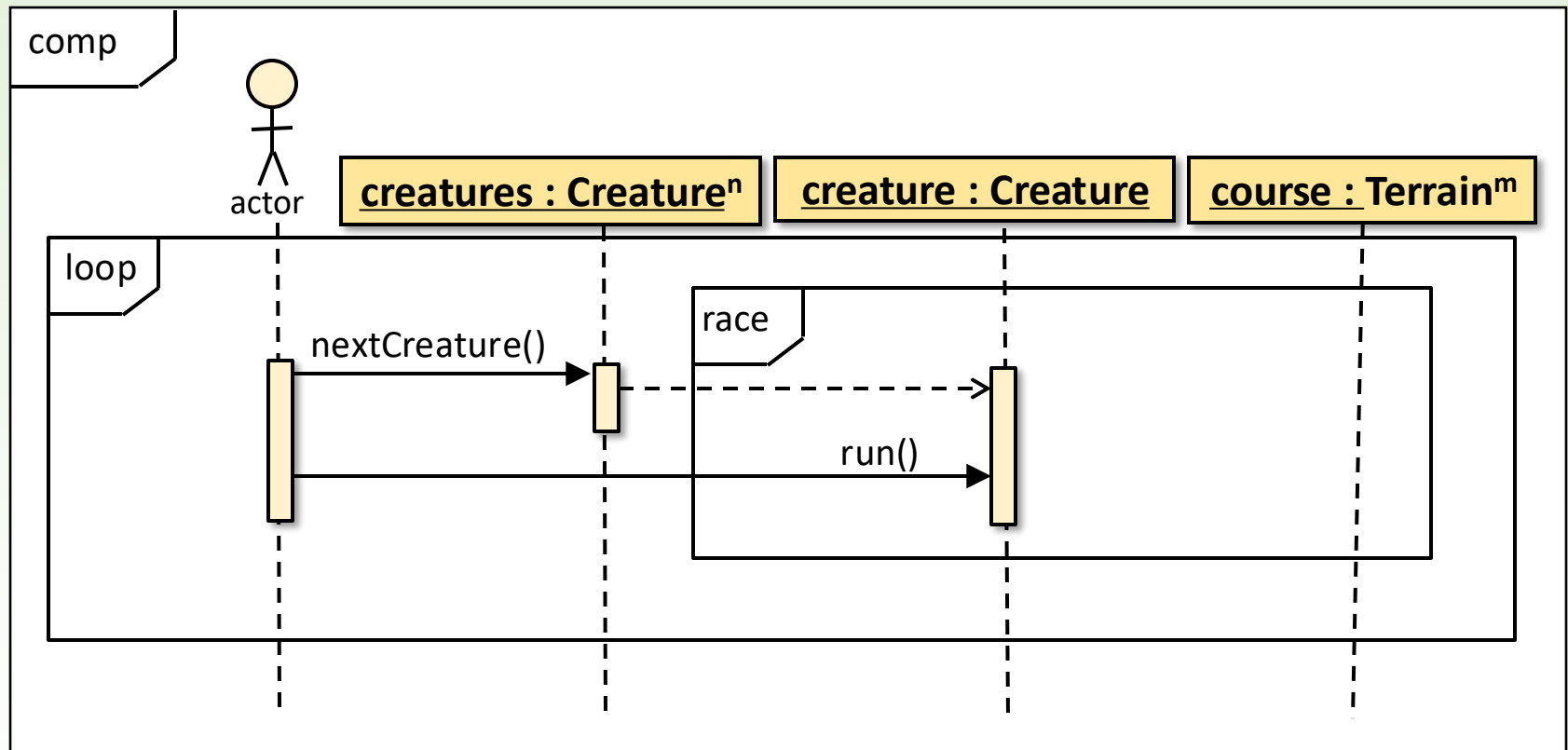
```

public class Greenie extends Creature {
    public Greenie(String name, int health) { super(name, health); }

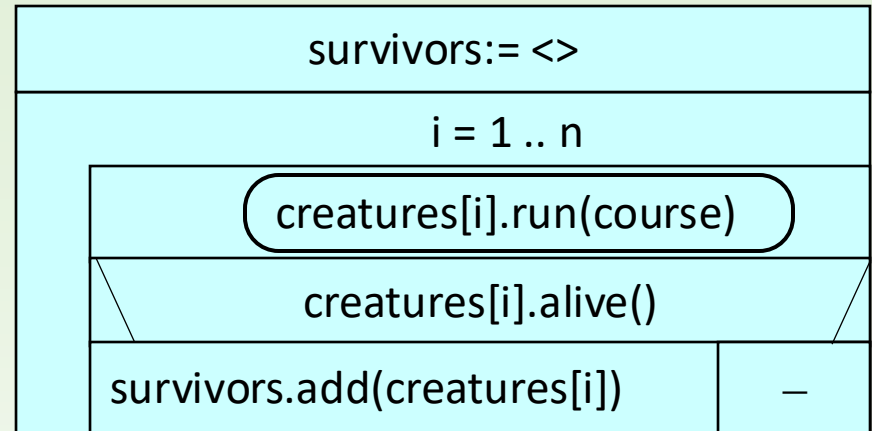
    @Override
    protected ITerrain traverse(ITerrain t) { return t.change(this); }
}

```

Competition of Creatures



Main program



Here the runtime polymorphism comes in handy: the behavior of race() - through the traverse() method - depends on what Species type the creature is, that is, what its class is.

```
// populating creatures
...
// populating courts
...
// competition

for (Creature creature : creatures) {
    creature.race(course);
    if (creature.isAlive()) {
        System.out.println(creature.getName());
    }
}
```

Creating creatures

input.txt

```
4
S plash 20
G greenish 10
D bug 15
S sponge 20
10
gmsgmgsgsm
```

```
int n = Integer.parseInt(reader.readLine());
List<Creature> creatures = new ArrayList<>();

for (int i = 0; i < n; i++) {
    String line = reader.readLine();
    if (line != null) {
        String[] tokens = line.trim().split("\\s+");
        char ch = tokens[0].charAt(0);
        String name = tokens[1];
        int p = Integer.parseInt(tokens[2]);

        Creature creature = switch (ch) {
            case 'G' -> new Greenie(name, p);
            case 'D' -> new Beetle(name, p);
            case 'S' -> new Splasher(name, p);
            default -> throw new IllegalArgumentException();
        };

        creatures.add(creature);
    }
}
```

```
// populating course
int m = Integer.parseInt(reader.readLine());
List<ITerrain> course = new ArrayList<>();
for (int j = 0; j < m; j++) {
    char c = (char) reader.read();
    ITerrain court = switch (c) {
        case 'g' -> Grass.getInstance();
        case 's' -> Sand.getInstance();
        case 'm' -> Swamp.getInstance();
        default -> throw new IllegalArgumentException("Unknown
ground type: " + c);
    };
    course.add(court);
}
```